QEDRank: Evaluation Methodology for Lean 4 Proof Benchmarks on ProofBench

Measuring instruments, validity controls and the state of the runs

Author names and affiliations

16 October 2026 (draft)

Abstract

QEDRank is a single-call evaluator for Lean 4 theorem-proving benchmarks, built around the private ProofBench corpus (111 problems at the commit used for the paper runs). It measures two tasks: *prove* (fill the `sorry` of a given formal statement) and *formalize* (write the statement from the paper, then a proof, with a mapping and confidence report). Every answer is compiled in the benchmark's pinned Lean/Mathlib project, checked for `sorry` and non-standard axioms, and classified into a fixed verdict set; provider failures (timeouts, rate limits, exhausted quota) are kept apart from model outcomes so they never inflate or deflate a score. Reported statistics are pass@1 with a paper-cluster bootstrap interval in the summary and a Clopper–Pearson interval in the LaTeX tables, the unbiased pass@k estimator of Chen et al. for repeated attempts, and McNemar tests with Holm correction for model comparisons.

As of 16 October 2026 no run yet yields reportable model performance. The 11 October pilot (Qwen3-30B-A3B via GWDG, Kimi K3 via Moonshot) was graded in the wrong Lean project and had a grader import bug; after re-grading, Qwen produced 8 compiling answers out of 109 and 0 verified proofs, and Kimi returned almost nothing because of account limits. The 12 October formalize run inherited the same project problem (65 of 84 reference statements unavailable). Harness fixes for both problems, pass@k support and provider-halt logic are implemented but uncommitted; the pass@8 runs (Kimi K3, Qwen3) and the self-hosted Qwen3 runs are prepared but not started.

Alongside QEDRank, the paper describes the complementary agentic harness of the ProofBench team (Inspect AI ReAct agent, up to 10 sessions with handoff notes, two-layer grading with `check_proof.py` and Lean FRO's Comparator), which was set up and verified on a Windows PC with Docker, and the three serving routes tried for an open-weight model: GWDG Chat AI (200 requests per hour), a vLLM server on the TU Freiberg HPC, and llama.cpp on a DGX-2 with V100 GPUs.

Contents

1 Introduction and scope **1**

2 Benchmark corpus **2**

3 Tasks **3**

4 Evaluation pipeline **3**

5 Models, providers and sampling **6**

6 Metrics and statistics **8**

7 Validity controls **9**

8 The agent harness (proofbench-eval-apptainer) **10**

9 Infrastructure **11**

10 The QEDRank application **13**

11 Runs performed to date **16**

12 Threats to validity and limitations **16**

13 Ethics and data handling **18**

14 Reproducibility **19**

1 Introduction and scope

The question is whether current language models can prove, or formalize and prove, research-level mathematics in Lean 4 when the only judge is the Lean kernel. ProofBench supplies the problems: statements taken from recent papers, formalized against Mathlib, kept private so that answers cannot be looked up. This paper describes the measuring instruments, not results.

Two instruments are used, on purpose, because they answer different questions (Table 1). QEDRank is the baseline and the statistics layer; the agent harness is the shared protocol of the ProofBench team. Both run the same problems from the same private statement repository, so a problem solved by the agent but not in one pass is attributable to tool use rather than to a different corpus.

Table 1: The two instruments.

	QEDRank	ProofBench agent harness (`proofbench-eval-apptainer`)
Protocol	one request per problem and attempt, no tools	ReAct agent with bash, Python, a Lean language server; up to 10 sessions chained by handoff notes
Question answered	what the model knows and can write down in one pass	what the model can achieve when it may compile, inspect errors and retry
Tasks	prove; formalize (statement + proof + mapping + confidences)	prove
Grading	compile in the pinned project, `sorry` and axiom check, statement-unchanged check; fidelity check of generated statements	layer 1 `check_proof.py`; layer 2 Comparator (kernel replay in a Landlock sandbox)
Statistics	pass@1 with intervals; unbiased pass@k; paper-cluster bootstrap; cost	accuracy ± standard error per run; per-problem `results.csv`
Author	this project	Utku Okur, ProofBench team

What this paper claims: the procedure is defined, implemented, tested on fixtures and live smoke tests, and its failure modes so far are known and recorded. What it does not claim: any score for any model. Section 11 lists every run to date and why none is reportable yet.

2 Benchmark corpus

All prove runs use the private repository `proof-bench/proofbench-statements` pinned to one commit; every run folder records that commit, so the problem set of a run is reproducible to the byte. Four snapshots have been used or compared (Table 2).

Table 2: Corpus snapshots.

Repository and ref	Commit	Date	Problems	Used for
`proofbench-statements`	`466b32a`	2026-10-08	111 main-theorem targets (P001–P111) plus 28 disproof variants, not evaluated	QEDRank pilot and re-grade
`proofbench-statements`	`0a4a476`	2026-10-11	114 (adds P112–P114, matroid intersection); P008 renamed, DISPROVE files for P057/P058	agent harness copy on the PC; P109–P111 byte-identical to `466b32a`
`vaibhav-baj/ProofBench` `main`	`1fa2abc`	2026-09-30	108; 102 identical to the above, 6 since changed (P001, P004, P008, P027, P067, P090), P109–P114 absent	older mirror, not used for runs
`vaibhav-baj/ProofBench` `Proof_Bench_Prototype`	`e631e6a`	2026-10-10	29 paper folders, 28 with a PDF; 3 targets each (main theorem, upper bound, lower bound)	formalize run

Problem layout. A challenge file `ProofBench/PROBLEMS/Pxxx/Pxxx.lean` imports only Mathlib and the statement layer `ProofBench.Mathlib_Problems.*`; its main theorem ends in a single `sorry` (three files, P015, P017 and P070, contain several). The proctor material (`ProofBench/Mathlib/`, `_UB`, `_LB`, `_sol`, `_DISPROVE` files) never reaches a model, in either instrument; the agent harness's image builder refuses any module outside the statement layer.

Claim status at `466b32a`. 80 main theorems are established, 30 are open problems and 1 is uncertain. Open and uncertain claims stay in the *statement track* (can the model state it) and leave the *proof track* (can the model prove it), because a verified proof of an open claim needs a human audit before it counts.

Pinned Lean project. Lean v4.33.1 with Mathlib `0df444a`, the ProofBench Matroid fork at `fa6c155`, and `maxSynthPendingDepth = 3`. The pilot was first graded in QEDRank's default project (Lean 4.34.1), which does not contain the benchmark's modules; that mistake and its correction are in Section 11.

Pilot set. P109, P110 and P111 (added 2026-10-08) are the three problems used for end-to-end tests of the agent harness; the sandbox image on the PC contains only their statement modules.

3 Tasks

QEDRank defines two tasks with one primary endpoint each. In both, a compiling file is never a success by itself.

Prove (endpoint `verified_pass1_prove`). Input: the challenge file with its `sorry`. Output: the completed file. Verified means the file compiles under the pinned toolchain, contains no `sorry`, keeps the main theorem and every challenge definition unchanged as compiled declarations, depends only on `propext`, `Classical.choice` and `Quot.sound`, and replays through the kernel.

Formalize (endpoint `verified_pass1_main_theorem`). Input: the target name and up to 24,000 characters of passages extracted from the paper's PDF. Output, in one reply: (A) a Lean statement of the claim with any definitions it needs; (B) a complete Lean proof file, or no file if the model cannot prove the claim; (C) a paper-to-Lean mapping; (D) two confidences from 0 to 100, that A is faithful and that B passes the audit. Part A is judged against an audited reference statement (Section 7); Part B is graded like a prove answer. Three targets per paper: main theorem, upper bound, lower bound.

Where a request can end. Every item record carries exactly one terminal category (Table 3). The categories are ordered so that the first ones are provider or grader failures and never enter a rate's denominator.

Table 3: Terminal categories.

Prove (per attempt)	Formalize Part A (statement)	Formalize Part B (proof)
request timed out	provider timeout / rate limit / quota	no proof file returned
rate limited (HTTP 429) or quota exhausted	grader environment failure	cut off at the token limit
abstained: no Lean file returned	no statement: cut off at the token limit	model declined to prove
cut off at the token limit (a failed attempt)	no statement: reply not in the required format	compile error
Lean compile error	statement imports a module that does not exist	compiled with `sorry`
Lean compile time limit reached	statement does not elaborate	statement changed or absent
compiled; target statement changed or absent	statement unchecked (no reference) / unresolved / different / faithful	verified
compiled; proof contains `sorry`		
verified		

Eligibility. *Statement track*: the paper × target has an audited reference statement (file present, target declaration resolved, reference elaborates); open claims included. *Proof track*: statement-track eligible and the claim is established (or refuted, for disproof targets with confirmed polarity). Infrastructure failures are excluded from both; abstentions count as failures in the proof track.

4 Evaluation pipeline

A run is seven stages that write into one folder; every stage can be resumed, and every number in the summary traces back to one item record on disk. The CLI is `node qedrank/cli.js run|resume|grade|stats|export|import-agent|compare|serve`. Figure 1 shows where one prove attempt can end; Table 4 lists the stages.

Item record (`items/<paper>__<target>__<provider>/attempt-a{k}r{r}.json`, schema `qedrank-item-v1`). It holds the generation (provider, model id, host, response model, finish

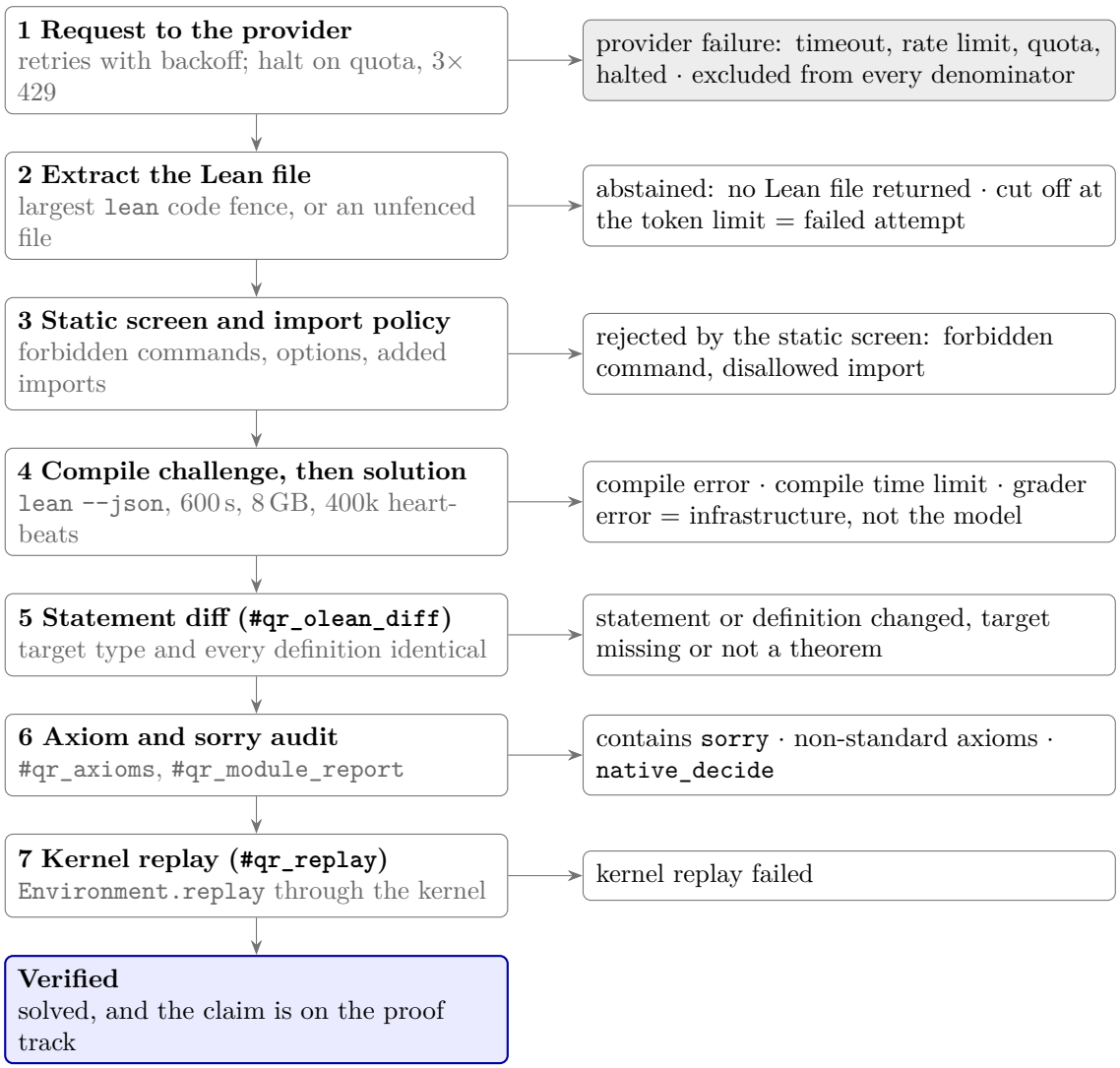


Figure 1: One prove attempt: seven checks, seven exits, one success. The first exit (tinted) is a failure of the infrastructure and leaves every denominator; from the second exit on, the attempt counts against the model. A grader error at step 4 is also infrastructure.

Table 4: Stages of a run.

Stage	What happens	Written
discovery	the corpus is fetched from GitHub or a local path at the pinned ref; the resolved commit is locked, and a later stage aborts if the repository's commit has moved	`manifest.json`
preparation	*prove*: the challenge file `challenge__main_theorem.lean` is copied and any file whose `sorry` count is not 1 is flagged. *formalize*: PDF passages are extracted, a definitions-only prelude is built with `lake build`, the reference statement is audited and compiled as its own module, and a per-target context is assembled; a leak guard rejects any prompt that contains reference proof text	`prepared/<paper>/`
generation	one request per attempt, prompt template `qedrank-prove-v1` or `qedrank-prompt-v2`; the prove loop runs attempt → problem → provider, so attempt 1 of every problem exists before attempt 2 starts (the formalize loop is paper → target → provider → attempt); before each live request the grader compiles the challenge itself, and a challenge that does not compile is recorded as an infrastructure failure, never charged to the model	`items/<key>/ response-a{k}r{r}. txt`, generation record
compilation	the model's whole file (no splicing) is extracted from the largest `lean` code fence, an unfenced file is accepted, no file is an abstention; the static screen and import policy run first; then `lean --json -M 8192 -DmaxHeartbeats=400000` is spawned directly with the project's `LEAN_PATH`, 600 s per file, two files at a time by default, and `*_API_KEY/GITHUB_TOKEN` removed from its environment	`.olean`, compiler output
audit	`#qr_olean_diff` compares the challenge's and the submission's object files: the target's type and every non-theorem declaration must be identical; `#qr_axioms` (via `collectAxioms`) and `#qr_module_report` find non-standard axioms and `sorry` in each declaration; `#qr_replay` re-checks the environment through the kernel	`grade` block of the item
comparison	*formalize* only: Part A is stripped of proofs and compiled in namespace `QRModel`; the statement-fidelity check of Section 7 compares it with the audited reference; Part B is compiled unchanged and its target checked for A/B consistency with `#qr_compare`	`comparison`, `statement_match`
statistics	item records are aggregated into results and the summary, with the paper-cluster bootstrap of Section 6; exports for the app	`results.json`, `summary.json`, `exports/items.json`, `progress.json`

reason, provider attempts and retry waits, usage in input / output / reasoning / cached tokens, observed latency including retries and working time of the successful request, the sampling actually sent, prompt hash and template version), the parse result, the static-screen and compile results, the audit, the statement match, timing, eligibility, the five outcome flags (`infrastructure_failure`, `compiled`, `audit_clean`, `statement_faithful`, `verified`) and free-text notes such as `output_truncated`, `multiple_blocks_largest_used`, `no_answer_within_output_budget` or `ESCALATE` (an apparent proof of an open claim). A grader error turns the item into an infrastructure failure. Cost is not stored per item; it is computed at summary time from the pricing table.

Run record (`run.json`, schema `qedrank-run-v1`): run id, source request and resolved commit with date, models with their hosts and facts, protocol (`single_call` or `agent`), task, harness commit with branch, dirty flag and changed files, prompt template version, system prompt, machine, the full effective config, stage records with counters (total, done, skipped, halted, generated, generation failed, compiled, verified, infrastructure, errors), provider halts, re-evaluations and wall-clock time. `resume` rebuilds the config in the order defaults < stored config < `--config` file < flags, restores the Lean project, skips completed attempts unless `--reevaluate`, and retries infrastructure failures with `--retry-failed`; pricing and bootstrap settings always come from the current config.

Agent protocol. QEDRank also contains its own Inspect AI harness (`harness/`: sandboxed agent, at most x sessions joined by a `HANDOFF.md`, QEDRank's `grade` after every session) whose Docker image is still untested, and the `import-agent` command, which turns an agent bundle, from that harness or from the ProofBench team's, into a run with protocol `agent` so the same statistics apply.

5 Models, providers and sampling

Two models have been run so far; the roster has eleven default slots behind one call path, so a further model is a configuration entry, not code. All slots except the Anthropic ones speak the OpenAI-compatible chat protocol, which is also what vLLM and llama.cpp serve (Table 5).

Sampling. Single-attempt runs use temperature 0 where the provider allows it. Pass@k needs temperature above 0; the Qwen3-2507 model card recommends 0.7 with top-p 0.8, top-k 20 and a presence penalty of 0–2, which `configs/qwen-prove-pass8.json` adopts. Whatever is sent is recorded in the generation's `sampling` field and in `summary.settings` (temperature, top_p, top_k, presence_penalty, effort, max_tokens). An environment variable `QEDRANK_EFFORT_<slot>` can override the configured effort for one run.

Retries, quota and halts. HTTP 408, 425, 429, 5xx and network errors are retried (4 times by default, 6 in the pass@8 configs) with delay min(max, Retry-After or base $\cdot\, 2^{a-1}$ + jitter); for 429 the wait is never shorter than the exponential term. A 429 whose body reports an exhausted balance becomes `insufficient_quota` and is not retried. Three consecutive rate-limit failures, or one quota failure, halt the provider for the rest of the process: later requests return `halted` without being sent and leave no item record, so the run can be resumed with `--retry-failed` once the account is restored. Error kinds: `not_configured`, `retired`, `http_error`, `rate_limited`, `insufficient_quota`, `timeout`, `network`, `empty_response` (finish reason `length` with no text), `refusal`, `provider_error`, `halted`, `challenge_does_not_compile`.

Table 5: Model roster.

Slot	Model id	Host	Request shape
`qwen`	`qwen3-30b-a3b-instruct-2507` (30.5 B parameters, 3.3 B active, released 2025-07-30)	`chat-ai.academiccloud.de/v1`; a base-URL override points it at a self-hosted server	`temperature`, and `top_p`, `top_k`, `presence_penalty` when configured; `max_tokens`
`kimi`	`kimi-k3` (2.8 T parameters, 104 B active, released 2026-07-16)	`api.moonshot.ai/v1`	no temperature (the provider sets it); `reasoning_effort` low, high or max; the model always reasons, and reasoning tokens count against `max_tokens`
`fable`, `opus`	`claude-fable-5-1`, `claude-opus-5-5`	Anthropic Messages	`output_config.effort`, no temperature
`gpt6_astra`, `gpt6_sol`, `gpt56_terra`, `gpt54`	`gpt-6-astra`, `gpt-6-sol`, `gpt-5.6-terra`, `gpt-5.4`	`api.openai.com/v1`	`max_completion_tokens` and `reasoning_effort`, no temperature
`llama`, `mistral`, `deepseek`	`meta-llama-3.1-8b-instruct`, `labs-leanstral-2603`, `deepseek-v4-pro`	GWDG, Mistral, DeepSeek	temperature sent
optional	any OpenAI-compatible endpoint (`custom`, OpenRouter, Together, Groq, NVIDIA, Hugging Face, Gemini)	by configuration	as above

Table 6: Prepared pass@8 configurations.

Setting	`kimi-prove-pass8.json`	`qwen-prove-pass8.json`
attempts	8	8
effort / temperature	max / provider default	— / 0.7, top-p 0.8, top-k 20
max output tokens	65,536	16,384
request timeout	2,700 s	600 s
retries, base, max delay	6, 15 s, 120 s	6, 15 s, 120 s
requests in flight per provider / items / Lean	8 / 16 / default	3 / 8 / 3
pricing (USD per M tokens, input / output)	3 / 15	none (academic endpoint)

What is recorded per request. Provider, model id, host, the model name the server reports, finish reason, number of provider attempts and total retry wait, input / output / reasoning / cached tokens, observed latency including retries, working time of the successful request, the sampling sent, prompt hash and template version. Cost is computed at summary time as input tokens × input price plus output tokens × output price, with prices keyed by model id.

6 Metrics and statistics

Every rate is a fraction num / den over eligible items, reported with its denominator; `summary.json` carries bootstrap intervals, the LaTeX matrices Clopper–Pearson intervals, and both say which.

Rates per model and target (`by_target[target][model]`): `statement_faithful`, `compile`, `audit_clean`, `verified_pass1`, `abstention`, `unresolved`, `timeout`; distributions of time (observed model latency, Lean time, total item time), tokens, lines of code; `cost`; `calibration`. Each rate field holds `num`, `den`, `value`, `ci_lo`, `ci_hi`, `ci_method`, `ci_B`.

Intervals. The summary uses a percentile bootstrap [5] that resamples *paper ids* with replacement, so all targets, models and attempts of one paper move together: $B = 2{,}000$, seed 20260930 (mulberry32), $\alpha = 0.05$, bounds null when fewer than 10 finite draws exist. The LaTeX generators and the benchmark comparison use the exact two-sided 95 % Clopper–Pearson interval [4], found by bisection on the binomial distribution; it treats items as independent and is the conventional interval for a benchmark table.

pass@k. For an item with n attempts of which c succeeded, the unbiased estimator of Chen et al. [3] is

$$\text{pass@}\,k \;=\; 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \;=\; 1 - \prod_{i=n-c+1}^{n} \Big(1 - \frac{k}{i}\Big), \tag{1}$$

computed as the product for numerical stability; it is undefined (null) when $n < k$ and 1 when $n - c < k$. The curve is reported at $k = 1, 2, 4, \ldots$ up to n and at n itself, each point with `items`, `items_short` (items with fewer than k attempts, which are left out of that point), `items_solved_any` and a bootstrap interval. The pool is the pass@1 setting, repair round 0, no infrastructure failure, proof track. The summary also gives the mean success per attempt, the per-attempt rates and their standard deviation across attempts, so a drift between attempt 1 and attempt 8 (a provider that degrades, a cache that warms) is visible.

Model comparison on the papers all models share: Cochran's Q [8] across models, exact McNemar tests [7] for each pair with Holm correction [6], a paired bootstrap of the difference, and a descriptive ranking by point estimate; only pairs listed as *statistically supported* are called different. No weighted composite score is computed.

Calibration. The two confidences of the formalize task (statement faithful, proof passes) are scored by root-mean-square error over 10 bins against the observed rate, with the abstention rate beside it.

Cost. Input tokens/10^6× input price + output tokens/10^6× output price per item, summed per model, and divided by the number of verified successes for *cost per verified success*; `n_priced` says how many items had a price.

Counting rules.

- A provider failure (timeout, rate limit, quota, halt) or a grader error is an infrastructure failure: excluded from every denominator and counted separately.
- An answer cut off at the output limit, including one whose budget was spent on reasoning with no visible text, is a failed attempt of the model.
- An abstention (no Lean file) is a failure in the proof track and is also reported as a rate of its own.
- A compiling file with `sorry`, a changed statement, a non-standard axiom or a failed kernel replay is *compiled* but not *verified*.
- Open and uncertain claims are counted in the statement track only; an apparent proof of one is flagged `ESCALATE` for human audit and does not enter `verified`.

7 Validity controls

Each control below exists because its absence was observed to bias a run at least once; Section 11 records the incidents.

Corpus and prompt integrity.

- The corpus commit is resolved at discovery and locked; a later stage aborts if the repository has moved.
- The formalize prompt passes a leak guard that rejects any text found in a reference proof; the prove prompt contains the challenge file and nothing else from the repository.
- Before a live request, the grader compiles the challenge file itself; a challenge that does not compile in the grader is an infrastructure failure, not a model failure.

Provider failures kept apart from model outcomes.

- Timeouts, rate limits, exhausted quota and halts are their own error kinds and never enter a denominator.
- A provider is halted after one quota error or three consecutive rate-limit errors; halted items are left unrecorded so that `resume --retry-failed` fills them later and a throttled run cannot yield a score.
- Attempt-major order in the prove task: attempt 1 of every problem exists before attempt 2 begins, so a run stopped early is a complete pass@1 run rather than a biased pass@k sample.

Grader strictness.

- Static screen: file size above 512 KiB, `prelude` or module headers, forbidden commands (`#eval`, `#exit`, `run_cmd`, `run_elab`, `run_tac`, `initialize`, `elab`, `macro`, `syntax`, `implemented_by`, `extern`, `unsafe`, ...), the option `debug.skipKernelTC`, heartbeat caps above the grader's.
- Import policy: the challenge's own imports are always allowed; added imports must start with Mathlib, Std, Lean, Init, Aesop, Batteries, Qq, ProofWidgets, Plausible, LeanSearchClient, ImportGraph or `ProofBench.Mathlib_Problems`; `ProofBench.Mathlib`, `ProofBench.PROBLEMS` and `ProofBench.DATABASE` are never allowed; dropping a challenge import is a rejection.
- Object-file diff of the target's type and every non-theorem declaration, `collectAxioms` against the allowed set, a per-declaration `sorry` report, and a kernel replay of the whole environment.
- Resource caps per compile (600 s, 8 GB, 400,000 heartbeats) and an environment stripped of API keys and the GitHub token.

Protocol stability.

- `resume` restores generation, Lean, grading and concurrency settings from `run.json`; the model list comes from the run's selection and the task from the run.
- Every run records the harness commit and whether the working tree was dirty, the prompt template version, the system prompt and the machine; the ProofBench harness freezes the same in `manifest.json`.
- Re-evaluations with a changed grader are appended to `run.reevaluations`, so a re-grade is visible, not silent.

Statement fidelity (formalize, Part A). The model's statement and the audited reference are compiled as separate modules and compared by `#qr_compare` at syntactic, reducible and default transparency; if not definitionally identical, both implications are attempted with a tactic portfolio (`exact`, `simpa`, `simp_all`, `aesop`, `omega`, `exact?`, optionally `unfold ...; aesop`), each proof must use its hypothesis, and neither statement may be provable outright. Outcomes: `definitionally_identical`, `equivalent`, `unresolved`, `not_evaluated`; `stronger`, `weaker`, `different` and `expert_confirmed_equivalent` come only from a human decision file (`review/decisions.json`). Accepted matches are the first two and the expert-confirmed one. An `unresolved` item is a question for an expert, never evidence of a mistranslation.

Related tool, not part of QEDRank. `QED_LEAF` (`qed-leaf equiv`, `qed-leaf fidelity-bench`, PR #2 of 2026-10-11) tests the opposite risk, that an equivalence checker accepts a wrong statement: it uses logic-level tactics only, because any two true closed statements are provable from each other, and takes ground truth from mutants that are refuted. On 24 core-Lean theorems with 279 variants it raised 0 of 96 false alarms on rewordings (against 96 for a structural and 57 for a definitional-equality check) and flagged all 93 refuted mutants. It has not yet been run against ProofBench's Mathlib project.

Human audit. Apparent proofs of open claims (`ESCALATE`) and every agent solve are reviewed by hand; a `review/audit.json` can reject a solve after the fact.

8 The agent harness (proofbench-eval-apptainer)

The ProofBench team's harness gives every model the same agent loop, tools, sandbox and budgets, and grades with Lean alone. It was copied from `proof-bench/proofbench-eval-apptainer` main (`2803e81`, 51 commits) to branch `DGX_Testing` of `vaibhav-baj/QEDRank` on 2026-10-13, and set up at `C:\proofbench-eval` on the PC.

Agent. An Inspect AI [10] ReAct [9] agent with `bash`, `python`, a text editor and `lean-lsp-mcp` (Lean language server as tools), optionally a web-search tool (off in all runs so far). The sandbox is a Docker container built from the pinned toolchain, the Mathlib cache, only the statement modules the selected problems import, and offline helpers (sympy, networkx, python-sat, SageMath 10.9 with GAP and nauty, CaDiCaL 1.7.3). The container has no network and no API key; the model never sees the proctor library.

Grading runs in a second `verifier` container the agent never touched; only the submission's source text crosses over. Layer 1, `check_proof.py`: statement unchanged, imports limited to Mathlib and the statement layer, no `sorry` or `axiom` in any file, helper files under `Submission/` build, main file compiles, axioms limited to `propext`, `Classical.choice`, `Quot.sound`; it also refuses `#` commands, `initialize`/`run_cmd`, kernel-skipping options and notation or scope changes

Table 7: Budgets (defaults of `scripts/run.sh`).

Limit	Value
Sessions per problem, chained by a handoff note	10
Output tokens per session, reasoning included	30,000
Total tokens per session	1,000,000
Working time per session	1,800 s
Output tokens for the handoff note	8,000
Tokens per model reply	16,000
Request timeout / single attempt	1,800 s / 600 s
Wall clock per problem (sessions × working time × 3)	54,000 s
Problems in parallel	4 (2 on the PC)
Epochs	1, fixed: a problem is never restarted from scratch

ahead of the statement. Layer 2, Comparator [18] (Lean FRO): rebuilds statement and submission in a Landlock [17] sandbox (`landrun`), requires identical statements and dependencies, replays the proof through the kernel via `lean4export` [19], and checks the axioms.

Outputs. `manifest.json` freezes harness commit, image id, Lean toolchain, ProofBench commit, model, effort, search, network, epochs and all limits; `scripts/analyze.py` writes `results.csv` with, per problem: `correct`, `sessions_used`, `session_ends` (submit, token, working, error), tokens in/out/total, total and working time, tool calls and errors, model calls, proof lines, compile seconds, axioms. Inspect reports accuracy ± standard error over the problems of a run.

Verification on the PC (2026-10-13). The P110 reference solution passed both layers ("Your solution is okay!"); a file that leaves the `sorry` was rejected (`sorryAx`). Image checks: Lean 4.33.1, P109 compiles, `lean-lsp-mcp` answers, verifier binaries present, Landlock engages in best-effort mode under WSL2.

Relation to QEDRank. `correct = 1` corresponds to QEDRank's *verified*; the harness measures one agentic attempt per problem, so pass@k is not defined for it, and provider failures appear as `session_ends = error` rather than as a separate class. QEDRank's `import-agent` command reads Inspect logs into its item-record format so that the same statistics and LaTeX tables can be produced for agent runs.

9 Infrastructure

The limiting resource was never compute but access: rate limits on hosted APIs, two-factor logins on the HPC, and GPU generations too old for current serving software. Four environments were used (Table 8).

Moonshot (Kimi K3). Tier 0 allows 1 concurrent request, 3 requests per minute and 1.5 M tokens per day; Tier 1 ($10 cumulative top-up) allows 15 concurrent requests with no daily cap. Price $3 / $15 per million input / output tokens. The model always reasons; `reasoning_effort` is low, high or max. Open weights exist (K3 1.56 TB MXFP4; K2.5 and K2-Thinking 595 GB INT4) but exceed every GPU budget available here.

Table 8: Environments.

Environment	Role	What worked	What failed, and the fix
Windows 11 PC, Docker Desktop (WSL2 kernel 6.18, Landlock ABI v7), Git Bash	runs QEDRank (CLI, run server, app) and the agent harness	both harnesses end to end; Lean compiles in 30 s–5 min per file	Python defaults (cp1252, CRLF) corrupted `lake-manifest.json` and `modules.txt` in the image build: fixed with `PYTHONUTF8=1` and explicit `encoding/newline` in `build_image.sh` (patch not yet upstreamed); Git Bash rewrote `/opt/...` paths: `MSYS_NO_PATHCONV=1`
GWDG Chat AI (`chat-ai.academiccloud.de`, OpenAI-compatible) [25]	hosted open-weight models	`qwen3-30b-a3b-instruct-2507` reliable, tool calls correct	key limits 30/min, 200/h, 1,000/day, 3,000/month make an agent run (hundreds of calls per problem) impractical; `qwen3.8-27b` returned HTTP 500 on tool calls; served precision not documented
TU Freiberg HPC (`llogin01`, Slurm, partition `gpu32long`, NVIDIA RTX PRO 5000 Blackwell 48 GB)	self-hosted vLLM [15] server for Qwen3-30B-A3B in BF16	vLLM 0.31.0 in a venv on system Python 3.12; job 6847 on 2 GPUs reached *Application startup complete* with 131k context and the `hermes` tool parser	compute nodes lack `Python.h`, so Triton's JIT compile failed: fixed by pointing `CPATH` at headers from a `uv`-managed CPython 3.12 in `$SCRATCH`; every login needs password + 2FA, port 22 to GitHub is blocked (`ssh.github.com:443`); the HPC-native sandbox needs the private `inspect-apptainer` package, not available to this account
DGX-2 (`dgx-2`, Ubuntu 22.04, kernel 5.15, 16 × Tesla V100 32 GB, shared)	run model server and harness on one machine, no tunnel, no PC	Docker with GPUs as an unprivileged user; `/raid/mitarbeiter/<user>` for the 61 GB model; `tmux`	`vllm/vllm-openai:latest` (torch 2.13, CUDA 13) has no `sm_70` kernels: "no kernel image is available"; llama.cpp [16] built from `ceefd16` with CUDA 12.8 for arch 70 answered a tool-call test with Qwen3-0.6B (GPU use still to be confirmed); V100 has no BF16, so the model runs in FP16; Python 3.10 is below the harness's 3.11 minimum (`uv` Python 3.12)

Serving routes compared. The hybrid route (vLLM on the HPC, harness on the PC through `ssh -N -L 8000:<node>:8000`) works but needs three open terminals and two 2FA logins per run. The DGX route needs one login and survives disconnects, at the price of FP16 and a different server program; runs are labelled with the server, precision and version (`selfhosted-bf16-vllm0.31.0`, `fp16-llamacpp-<commit>`) so that the paper can report them apart.

10 The QEDRank application

The web app is the operator's console: it freezes a corpus, starts and watches runs, and renders the summary a run produced. It computes no statistic itself (the Evaluation page states "Nothing is computed here"); every number on screen is read from `summary.json`, so a figure in the app and a table in the paper come from the same file. Figure 2 shows the parts; Table 9 the six pages and Table 10 the sections of the Evaluation page.

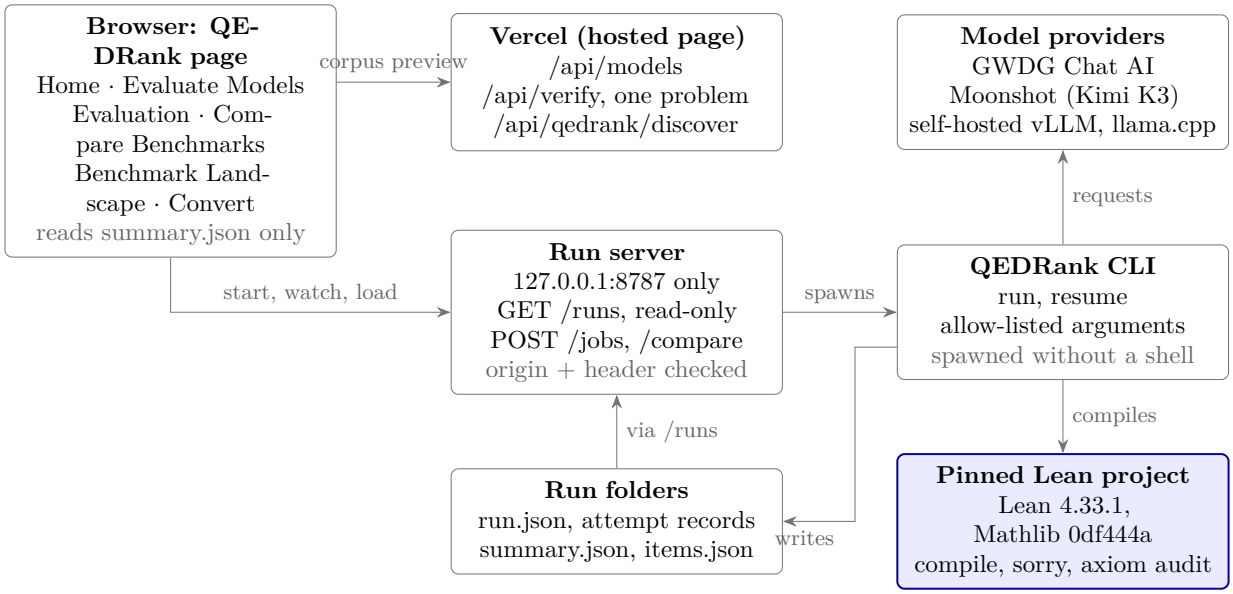


Figure 2: QEDRank architecture. Scores come from the local CLI and Lean; the hosted page only previews. The browser only starts jobs and reads files; the run server spawns the CLI with validated arguments, the CLI calls the providers and the pinned Lean project, and writes the run folder that the page reads back.

Every figure has a Chart/Table toggle, an Enlarge view and an error boundary (`ChartBoundary`) that shows the table if a chart throws; the Evaluation page went blank on the Kimi runs until the pair matrix learned to render an empty cell when two models share no graded paper.

Run server (`node qedrank/cli.js serve`, or a double-click on `scripts\serve.cmd`). It binds to 127.0.0.1:8787 and serves run folders read-only (`GET /runs`, `GET /runs/<id>/<file>`, file types limited to json, csv, txt, lean, md, pdf, with a path-traversal check). `POST /jobs` and `POST /compare` answer only to the app's own origins (localhost, `*.vercel.app`, or `--allow-origin`) with the header `X-QEDRank-Client`, accept at most 64 KB, allow only `run` and `resume`, validate every argument against a regular expression or allow-list, spawn the CLI with an argument array (never a shell), run one job per run id, cap the log at 2 MB, and stop a job with SIGTERM then SIGKILL after 5 s. The GitHub token comes from `GITHUB_TOKEN` or `gh auth token`, reaches the child process only and is never logged.

Table 9: Pages of the app.

Route	Page	What the operator does there	Data it reads	Role in testing
`/`	Home	chooses one of the two workflows, Evaluate Models or Compare Benchmarks, each listed with what it measures	roster summary only	entry point; old links to the single-problem anchors are forwarded to Convert a Problem
`/benchmark`	Evaluate Models	step 0 picks models (with the masked key setup); step 1 *Freeze and audit the corpus* (repo link, ref, subfolder, expected paper count → manifest with commit, paper table, target counts, flags); step 2 *Run the benchmark from here* (task prove/formalize, targets, papers, run id, Lean project, dry run; Start, Stop, Load into Evaluation) or *Load a run* (list, load, auto-refresh, or open files)	`POST /api/qedrank/discover`; run server `POST /jobs`, `GET /jobs/<id>?from=N`, `POST /jobs/<id>/stop`, `GET /runs`, `run.json`, `progress.json`, `summary.json`, `exports/items.json`	the only place a run is started from the page; shows the equivalent CLI command; the progress view counts generated / compiled / verified / infrastructure / error items live
`/evaluation`	Evaluation	reads a loaded run; downloads Summary JSON, Matrix CSV, Items JSON and a LaTeX report (metadata, models, result tables, item outcomes)	`summary.json`, `items.json` from session storage	the analysis console (Table 10)
`/versus`	Compare Benchmarks	your benchmark (URL, ref, subfolder, optional token) against up to 7 benchmarks from `qedrank/data/benchmark-literature.json`; downloads JSON, Matrix CSV and LaTeX	run server `POST /compare`	produces the benchmark-comparison table and figures used in `paper/benchmark-comparison.tex`
`/compare`	Benchmark Landscape	ProofBench against the published Lean benchmarks: facts table, size against best reported success, statement complexity, subject heatmap, level composition, sources	`runs/_compare/benchmark-comparison.json` (schema `qedrank-benchmark-comparison-v1`) or a file	positions the corpus relative to miniF2F [21], PutnamBench [22] and others for the related-work section
`/convert`	Convert a Problem	types one problem, sends it to every selected model with a key, rates the Lean output 1–5	`POST /api/verify` (Vercel function), `POST /api/feedback`	model and key check before a run; correctness is not judged here

Table 10: Evaluation page: sections and the summary fields they read.

#	Section	Figure or table	Summary field
–	run bar and KPI tiles	run id, corpus commit, toolchain, harness commit, protocol, settings (k, temperature, max tokens, bootstrap B and seed); best verified pass@1, papers, items, models, Cochran's Q, supported pairs	`run`, `settings`, `coverage`, `model_comparison`
0	Agent protocol (agent runs only)	solved within k sessions, budgets, tool calls	`agent`
1	Leaderboard	HBars of verified pass@1 with CI whiskers; GroupedBars of verified, statement faithful, compile, audit clean; one HBars per target; pass@k GroupedBars when $k > 1$	`by_target[scope][model]`, `pass_at_k.curve`
2	Heatmap	models × scopes, metric tab	`by_target`
3	Full matrix	16-column MatrixTable with sandwich and paper-level columns; eligibility rules	`by_target`, `paper_level`, `eligibility_rules`
4	Statement matches	StackedBars of faithful / different / unresolved / not evaluated	`statement_match_counts`
5	Time	RangeDots (median, IQR, p90) of model latency, Lean time, total item time	`time.*`
6	Tokens and cost	GroupedBars of median input/output tokens; HBars of cost per verified success	`tokens.*`, `cost.per_verified_success_usd`
7	Calibration	Reliability diagram per model: stated confidence against observed rate, statement and proof audit	`calibration.statement`, `calibration.proof_audit`
8	k-model comparison	descriptive ranking, PairMatrix of row-minus-column percentage points with • for Holm $p < 0.05$, McNemar table	`model_comparison`
9	Subgroups	HBars per subject and per difficulty	`subgroups.by_subject`, `by_difficulty`
10	Coverage	sandwich coverage, reference defects, subsets, caveats	`paper_level`, `coverage`, `reference_defects`, `caveats`
11	Items	8 filters plus search, per-item details, filtered CSV/JSON	`items.json`

Deployment. The page is hosted on Vercel (project `qedrank`); only the corpus preview and the single-problem bridge run there. Generation, Lean checking and evaluation always run on the operator's machine through the run server, which the deployed page reaches as a private-network request.

11 Runs performed to date

None of the runs in Table 11 is reportable as model performance; each row says why. Numbers come from the run folders' `summary.json` and the generated LaTeX matrices.

Lessons already applied. The grader compiles the challenge file first and judges only imports a solution adds (`5de6055`). `resume` restores the stored generation, Lean, grading and concurrency settings, so a resumed run cannot silently change protocol. An answer cut off at the output limit is a failed attempt of the model, not an infrastructure failure. A provider is halted after exhausted quota or three consecutive rate-limit errors, and halted items stay unrecorded for `resume --retry-failed`, so a rate-limited run never produces a score.

12 Threats to validity and limitations

The two faults that invalidated the first runs were both in the measuring instrument, not in the models; the list below separates what has been fixed from what remains a limitation of the design.

Construct validity.

- A compiling file is not a proof: 7 of Qwen's 8 compiling answers kept the `sorry`. Only kernel-replayed, axiom-checked, statement-unchanged files count. For formalize, an automated failure to prove two statements equivalent is not evidence that they differ; such items are *unresolved* and need expert review.
- The prove prompt contains the challenge file but not the definitions of the imported statement layer `ProofBench.Mathlib_Problems`, which 52 of 111 problems use. Kimi declined those problems. Adding the definitions changes the protocol, so it must be decided before the pass@k runs and applied to every model alike.

Internal validity.

- Grading in the wrong Lean project and a too-strict import policy (fixed in `5de6055`, verified by re-grading with 0 grader errors).
- 59 % of Qwen's answers were cut off at 16,384 output tokens, and at temperature 0 thirty formalize answers fell into repetition loops; the compile rate of such a run measures the budget as much as the model. Pass@k configs use 16,384 tokens at temperature 0.7 for Qwen and 65,536 tokens for Kimi; the token limit is reported next to every rate.
- Provider failures were 85 % of Kimi's pilot. They are excluded from denominators, but a model with few graded items has wide intervals and is not comparable; the provider-halt rule now stops a run instead of recording such items.
- A resumed run could once change its settings; `resume` now restores them from `run.json`.

External validity.

- One corpus of 111 problems clustered in papers; two models so far, both mixture-of-experts; the open-weight one is small (3.3 B active parameters). Results will not generalise to frontier proprietary models without running them.

Table 11: Runs to date.

Run	Date	Task, corpus, models	Status	Key numbers
`2026-10-11-466b32a`	2026-10-11	prove; 111 problems; Qwen3-30B-A3B (GWDG), Kimi K3 (Moonshot); temperature 0, 16,384 tokens, 1 attempt	invalid grading: QEDRank's default Lean project (4.34.1) instead of the benchmark's; grader rejected imports that the challenge file itself contains	48 Qwen + 2 Kimi answers failed with `unknown module prefix 'ProofBench'`; 11 Qwen answers wrongly rejected as `disallowed_import`
`2026-10-11-466b32a-regrade`	2026-10-12	the same stored answers, re-graded in the pinned project (grader fix `5de6055`)	grading valid; Qwen numbers are a single-attempt baseline, Kimi's are not comparable	Qwen: 111 answered, 66 cut off at the token limit, 8/109 compiled, 7 of them with `sorry`, 1 with a changed statement, 0/80 verified. Kimi: 94/111 provider failures (37 timeouts at 300 s, 57 HTTP 429), 15 abstentions, 2 graded, 0 verified
`2026-10-12-e631e6a`	2026-10-12	formalize; 28 papers × 3 targets; same models	reference statements prepared in the wrong project: 65/84 unavailable, 3 more lost to a QEDRank namespace-wrapper bug; only 16 main-theorem references; every prompt lacked the per-paper definitions	Kimi: 0 answers (14 HTTP 429, 70 refused for insufficient balance). Qwen: 0/77 statements compile, 35 cut off (30 of them repetition loops at temperature 0), 0/15 verified. Two counting bugs (comment-only proof files counted as compiled; non-existent Mathlib imports labelled infrastructure) are corrected in the LaTeX generator, not yet in the summary code
agent harness, GWDG	2026-10-13	prove, agent; P109–P111 at `0a4a476`; Qwen3-30B-A3B via GWDG	stopped after 14 min at the user's request, 0 problems finished (200 requests/hour)	`manifest.json` records `harness_dirty = true` (Windows build patch)
agent harness, self-hosted	2026-10-13	prove, agent; P109–P111; Qwen3-30B-A3B via HPC vLLM	aborted after 1 min: the SSH tunnel was not open, every model call would have failed	none
Kimi K3 pass@8 (`configs/kimi-prove-pass8.json`)	not started	prove; 111 problems; 8 attempts, effort max, 65,536 tokens, 45 min timeout	waiting on: account top-up to Tier 1, a written no-training confirmation from Moonshot, and the decision whether to add hidden definitions to the prompt	estimated cost $170–390 at effort max (pass@1 alone $21–49); estimate, not measured
Qwen3 pass@8 (`configs/qwen-prove-pass8.json`)	not started	prove; 111 problems; 8 attempts at temperature 0.7, top-p 0.8, top-k 20	live smoke test (2 problems × 2 attempts) passed end to end; waiting on the serving route	—

- The same model is served three ways (GWDG, vLLM BF16, llama.cpp FP16) with different precisions and server programs; runs carry the route in their id and are reported separately.
- The agent harness runs one attempt per problem and its sandboxes have reduced Landlock protection on WSL2 and on the DGX's 5.15 kernel; its `manifest.json` records `harness_dirty = true` while the Windows build patch is unmerged.

Statistical conclusion validity.

- Items are not independent; the summary's intervals come from a bootstrap that resamples papers, the LaTeX tables use binomial (Clopper–Pearson) intervals that ignore the clustering, and model comparisons use McNemar tests with Holm correction. Small denominators (8 eligible Kimi items) make most pairs unsupported.
- The unbiased pass@k estimator needs $n \geq k$ attempts per item; the summary reports the number of items with fewer attempts for each k.

Contamination. Neither provider publishes a training cutoff and the corpus does not record publication dates, so exposure to the papers or to public formalizations cannot be assessed; solves of open problems are audited by hand.

13 Ethics and data handling

The corpus is private and partly unpublished, so every model call is a disclosure; the procedure limits what is disclosed, to whom, and under which terms.

- **Unpublished statements.** Problems are sent only to providers whose terms exclude training on API input, or to servers under the team's control. Moonshot (Kimi) has not yet given a written no-training confirmation; the Kimi runs stay blocked until it does. GWDG Chat AI is operated by an academic computing centre; the self-hosted routes (HPC, DGX) disclose nothing outside the university.
- **Proctor material.** Reference proofs, bounds and disproofs never enter a prompt or a sandbox image; the image builder refuses modules outside the statement layer, and QEDRank's prove prompt contains only the challenge file.
- **Credentials.** API keys and the vLLM server key live in the user's environment or are read at a hidden prompt; they are never written to a tracked file, printed, or typed into a web page. The GitHub token comes from `gh auth token`. The run server binds to 127.0.0.1, answers job requests only to the app's own origins with a custom header, and runs an allow-list of commands with validated arguments, never a shell.
- **Sandboxing.** Agent containers have no network and no host directories; grading runs in a second container as an unprivileged user; Comparator adds a Landlock sandbox (best-effort on WSL2 and on the DGX's 5.15 kernel, which are below the ABI level it was designed for).
- **Shared hardware.** The DGX-2 is shared; GPU 8 was in use by another user during setup. The model (61 GB) and images go to the group folder `/raid/mitarbeiter/<user>` and to Docker's store on `/raid`, which was 96 % full; the administrator should be told before full-size runs.
- **Contamination.** Neither provider states a training cutoff and the problems' publication dates are not recorded in the corpus, so prior exposure cannot be assessed; this is reported as a caveat on every summary.

14 Reproducibility

A run is reproduced from its folder alone: `run.json` names the corpus commit, the harness commit, the Lean project and every setting; the item records hold the raw responses. Table 12 lists the locations; the commands below are the ones used.

Table 12: Locations.

Where	What
`C:\QEDRank-QEDRank`	QEDRank, branch `QEDRank`, HEAD `bfc8df8`; 21 modified and several untracked files hold the pass@k, halt and page fixes (uncommitted on 2026-10-16)
`C:\proofbench-statements`	the benchmark's pinned Lean project (Lean/Mathlib v4.33.1, all 185 modules built), used with `--lean-project`
`C:\proofbench-eval`	the agent harness (branch `DGX_Testing`, upstream `2803e81`) with `proofbench-statements` at `0a4a476`, image `proofbench-sandbox:latest` (P109–P111)
`runs/<run-id>/`	`run.json`, `manifest.json`, `prepared/`, `items/<key>/attempt-a{k}r{r}.json` and `response-*.txt`, `results.json`, `summary.json`, `exports/items.json`, `progress.json`
`paper/`	`make-tex.cjs` (prove runs, $k \geq 1$) and `make-tex-formalize.cjs` (formalize, $k = 1$) regenerate every table and figure from a run folder; outputs `performance-matrix*.tex/.pdf`, `benchmark-comparison.tex`

```
# prove, 8 attempts, Qwen via the configured base URL, graded in the pinned project
node qedrank/cli.js run --repo https://github.com/proof-bench/proofbench-statements \
  --ref 466b32a --task prove --targets main_theorem --models qwen \
  --lean-project C:\proofbench-statements --config configs/qwen-prove-pass8.json \
  --run-id 2026-10-xx-466b32a-qwen-pass8
node qedrank/cli.js resume --run-id 2026-10-xx-466b32a-qwen-pass8 --retry-failed
node paper/make-tex.cjs runs/2026-10-xx-466b32a-qwen-pass8 \
  paper/performance-matrix-qwen-pass8.tex
npm test     # 87 passed, 1 skipped on 2026-10-13
             # (Lean fixture tests need QEDRANK_LEAN_TESTS=1)

# agent harness (Docker machine)
IDS="P109 P110 P111" scripts/build_image.sh
IDS="P109 P110 P111" MAX_SAMPLES=2 \
  MODEL=openai-api/qwen/qwen3-30b-a3b-instruct-2507 scripts/run.sh
python3 scripts/analyze.py logs/<RUN_ID>     # results.csv
```

Frozen per run. Corpus commit and date; harness commit, branch, dirty flag and changed files; Lean toolchain, Mathlib revision and lake options; prompt template version and system prompt; model ids and hosts; temperature, extra sampling, effort, max tokens, timeout, attempts, repair rounds, concurrency; bootstrap B and seed; pricing used for cost. For the agent harness: image id, model, effort, search, network, epochs, sessions and all token and time limits.

Before the pass@k runs (open items found while writing this paper).

- ☐ Commit the uncommitted harness changes, so that runs stop recording `harness.dirty = true`
- ☐ Decide whether the prove prompt includes the definitions of `ProofBench.Mathlib_Problems` (52 of 111 problems); apply to all models
- ☐ Pass the pinned project's `leanOptions` (for example `maxSynthPendingDepth = 3`) in formalize compiles as `grade` already does

- ☐ Fix `settings.primary_endpoint` in `run.json`, which prints the formalize text for prove runs
- ☐ Record `requests_not_sent` per halted provider rather than stage-wide
- ☐ Align the web launcher's default task (`prove`) with the run server's default (`formalize`)
- ☐ Move the two formalize counting corrections (comment-only proof files; non-existent Mathlib imports) from the LaTeX generator into the summary code
- ☐ Moonshot: Tier 1 top-up and a written no-training confirmation before any Kimi request with unpublished statements
- ☐ DGX: confirm that llama.cpp uses the GPU, convert the official Qwen3-30B-A3B weights to FP16 GGUF, build the sandbox image, and label runs `fp16-llamacpp-<commit>`
- ☐ Upstream the Windows `build_image.sh` encoding patch to `proofbench-eval-apptainer`

References

[1] de Moura, L., Ullrich, S.: The Lean 4 Theorem Prover and Programming Language. In: Automated Deduction – CADE 28, LNCS 12699, pp. 625–635. Springer (2021). doi:10.1007/978-3-030-79876-5_37

[2] The mathlib Community: The Lean Mathematical Library. In: Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP 2020), pp. 367–381 (2020). doi:10.1145/3372885.3373824

[3] Chen, M., Tworek, J., Jun, H., et al.: Evaluating Large Language Models Trained on Code. arXiv:2107.03374 (2021). (unbiased pass@k estimator)

[4] Clopper, C. J., Pearson, E. S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika 26(4), 404–413 (1934)

[5] Efron, B.: Bootstrap methods: another look at the jackknife. Annals of Statistics 7(1), 1–26 (1979)

[6] Holm, S.: A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979)

[7] McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika 12(2), 153–157 (1947)

[8] Cochran, W. G.: The comparison of percentages in matched samples. Biometrika 37(3/4), 256–266 (1950)

[9] Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing Reasoning and Acting in Language Models. In: ICLR 2023. arXiv:2210.03629

[10] UK AI Security Institute: Inspect: an open-source framework for large language model evaluations (2024–). `https://inspect.aisi.org.uk`

[11] Qwen Team: Qwen3 Technical Report. arXiv:2505.09388 (2025)

[12] Qwen Team: Qwen3-30B-A3B-Instruct-2507, model card (2025). `https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507`

[13] Moonshot AI: Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 (2025). †

[14] Moonshot AI: Kimi K3, model card (2026). `https://huggingface.co/moonshotai/Kimi-K3`

[15] Kwon, W., Li, Z., Zhuang, S., et al.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: Proc. 29th Symposium on Operating Systems Principles (SOSP 2023). (vLLM)

[16] Gerganov, G., and contributors: llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, commit `ceefd16` (2026-10-16)

[17] Salaün, M.: Landlock: unprivileged access control. Linux kernel documentation. `https://docs.kernel.org/userspace-api/landlock.html`

[18] Lean FRO: Comparator: independent checking of Lean submissions against a challenge. https://github.com/leanprover/comparator

[19] Lean FRO: lean4export. https://github.com/leanprover/lean4export

[20] lean-lsp-mcp: Lean language server as MCP tools. https://github.com/oOo0oOo/lean-lsp-mcp †

[21] Zheng, K., Han, J. M., Polu, S.: miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In: ICLR 2022. arXiv:2109.00110

[22] Tsoukalas, G., Lee, J., Jennings, J., et al.: PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. In: NeurIPS 2024 Datasets and Benchmarks. arXiv:2407.11214

[23] Schmitt, J., Bérczi, G., Dekoninck, J., et al.: IMProofBench: Benchmarking AI on Research-Level Mathematical Proof Generation. arXiv:2509.26076 (2025). †

[24] Kurtzer, G. M., Sochat, V., Bauer, M. W.: Singularity: Scientific containers for mobility of compute. PLoS ONE 12(5), e0177459 (2017). (Apptainer's origin)

[25] GWDG / KISSKI: Chat AI, OpenAI-compatible API for hosted open-weight models. https://chat-ai.academiccloud.de

[26] ProofBench team: proof-bench/proofbench-statements (private statement corpus; commits 466b32a, 0a4a476) and proof-bench/proofbench-eval-apptainer (agent harness; commit 2803e81). Private GitHub repositories (2026)

[27] This project: QEDRank, vaibhav-baj/QEDRank, branch QEDRank (evaluator, app, LaTeX generators) and branch DGX_Testing (copy of the agent harness). Private GitHub repository (2026)

† Cited from memory without access to the publication page; verify the identifier before submission.